Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#4)

Use of Venus Simulator for Register Level Execution of RISC-V Arithmetic and Memory Instructions

Student Name: **AHMED JAVED QURESHI** Roll No: **24i-6554** Section: **CE-A**

Date performed: **8 September,2026**

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:
In this task, I used the **Venus Simulator** to perform basic arithmetic operations. I initialized A = 5 and B = 4 in different registers. Then I performed addition, subtraction, multiplication, and square of A. I stored each result in a separate register. Finally, I ran the code in Venus and checked the results in the registers.
Code:
.text .data .globl main addi s0, x0, 5 # A addi s1, x0, 4 # B add s2, s0, s1 # C = A + B sub s3, s0, s1 # D = A - B mul s4, s0, s1 # E = A * B mul s5, s0, s0 # F = A ^ 2 li a7, 93 li a0, 0 ecall **PART V:**

Instruction # 1: addi s0, x0, 5

Hex = 0x500413

Decimal = (3*16^0) + (1*16^1) + (4*16^2) + (5*16^5) = 5243923

Instruction # 2: addi s1, x0, 4

Hex = 0x400493

Decimal = (3*16^0) + (9*16^1) + (4*16^2) + (4*16^5) = 4195475

Instruction # 3: add s2, s0, s1

Hex = 0x940933

Decimal = (3*16^0) + (3*16^1) + (9*16^2) + (4*16^4) + (9*16^5) = 9701683

Instruction # 4: sub s3, s0, s1

Hex = 0x409409B3

Decimal = (3*16^0) + (11*16^1) + (9*16^2) + (4*16^4) + (9*16^5) + (4*16^7) = 1083443635

Instruction # 5: mul s4, s0, s1

Hex = 0x2940A33

Decimal = (3*16^0) + (3*16^1) + (10*16^2) + (4*16^4) + (9*16^5) + (2*16^6) = 43256371

Instruction # 6: mul s5, s0, s0

Hex = 0x2840AB3

Decimal = (3*16^0) + (11*16^1) + (10*16^2) + (4*16^4) + (8*16^5) + (2*16^6) = 42207923

Instruction # 7: li a7, 93

Hex = 0x5D00893

Decimal = (3*16^0) + (9*16^1) + (8*16^2) + (13*16^4) + (5*16^5) = 97519763

Instruction # 8: li a0, 0

Hex = 0x513

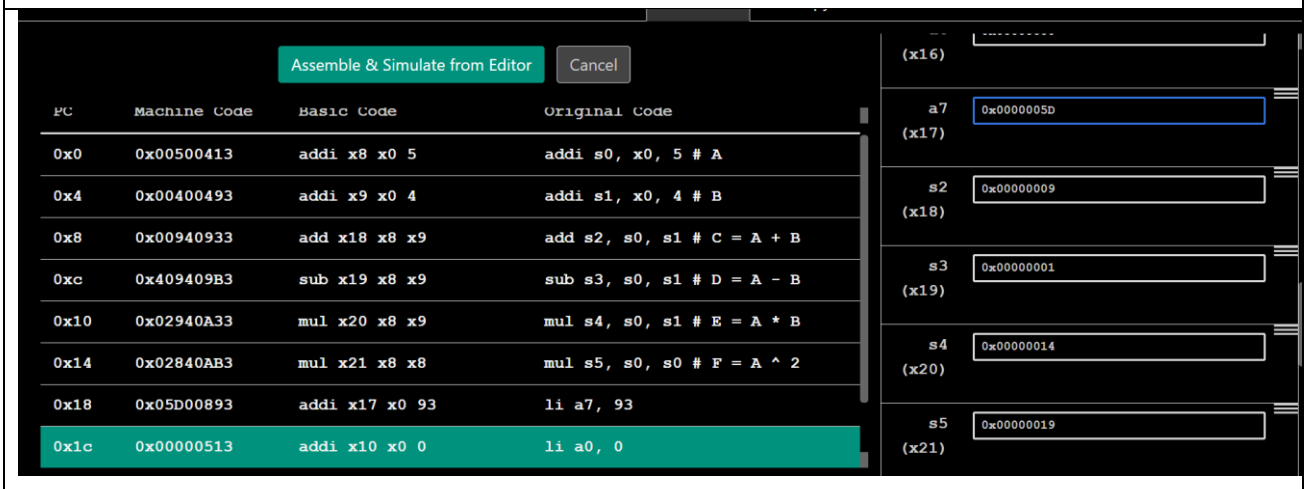
Decimal = (3*16^0) + (1*16^1) + (5*16^2) = 1299

Instruction # 9: ecall

Hex = 0x73

Decimal = (3*16^0) + (7*16^1) = 115

Output:



Task 2:

Description:

In this task, I converted the given mathematical expression into **RISC-V assembly language**.
I used registers to perform the required subtraction, addition, and multiplication.
I calculated the intermediate values step by step and stored them in registers.
The final value of m was obtained by combining these results.
I tested the complete program using the **Venus Simulator**

Code:

```
.text

.data

.globl main

 addi s0, x0, 5 # A

 addi s1, x0, 4 # B

 add s2, s0, s1 # C = A + B

 sub s3, s0, s1 # D = A - B

 mul s4, s0, s1 # E = A * B

 mul s5, s0, s0 # F = A ^ 2



 sub s6, s0, s2 # A - C

 add s7, s3, s4 # D + E

 mul s8, s7, s5 # (D+E)*F

 add s9, s6, s8 # m

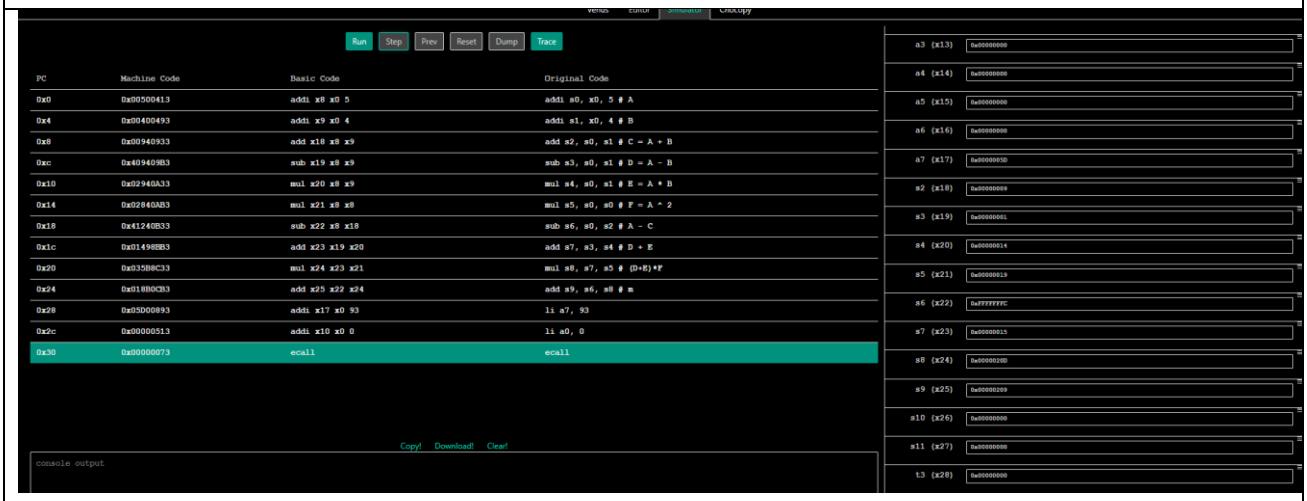


 li a7, 93
```

```
li a0, 0

ecall
```

Output:



Task 3:

Description:

In this task, I performed **AND, OR, and XOR** operations using RISC-V assembly language.
I used the hexadecimal values `0xFFF` and `0xF0F` as the input values.
Both values were stored in registers before performing the operations.
The results were stored in separate registers for easy observation.
I used **Venus Simulator** to run the program and verify the results.

Code:

```
.text

.data

.globl main

 li s0, 0xFFF

 li s1, 0xF0F



 and s2, s0, s1

 or s3, s0, s1

 xor s4, s0, s1

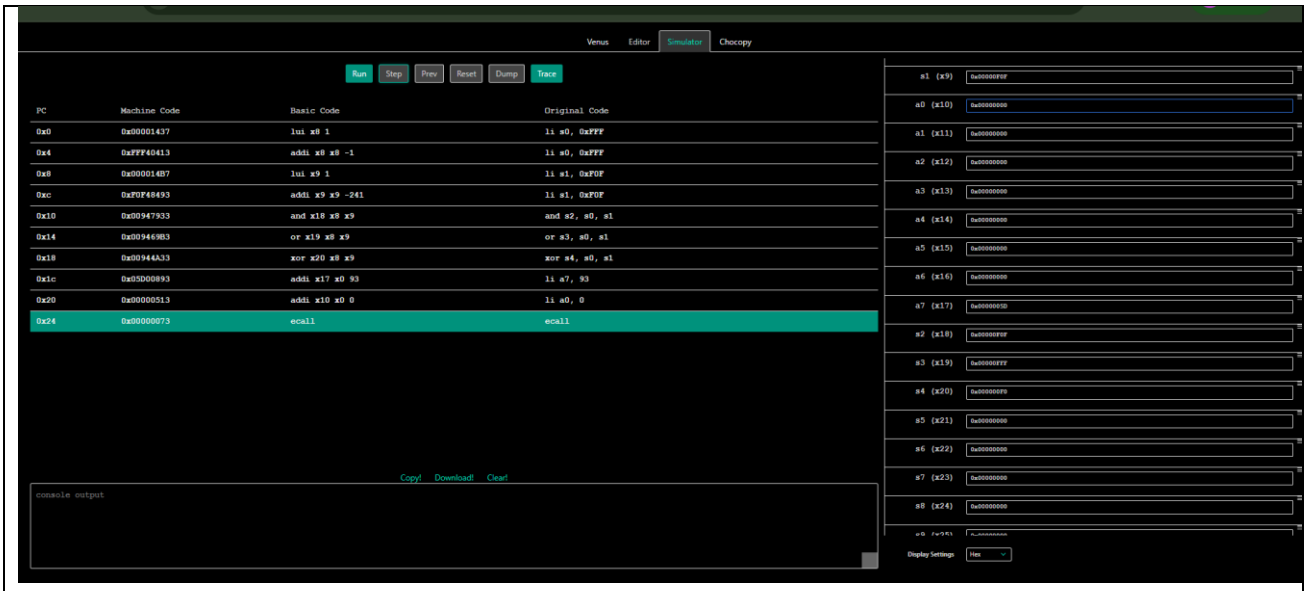


 li a7, 93

 li a0, 0

 ecall
```

Output:



Task 4:

Description:
In this task, I worked with an integer array containing the values **10, 20, 30, 40, and 50**. I used registers to store the array address, size, loop counter, and sum. A loop was used to access each array element one by one. I used the lw instruction to load the values from memory and add them together. Finally, I checked the calculated sum using the **Venus Simulator**.
Code:

```
.data

array: .word 10, 20, 30, 40, 50

.text
.globl main

main:
    la a0, array
    li a1, 5

    li t0, 0
    li t1, 0

loop:
    bge t1, a1, done

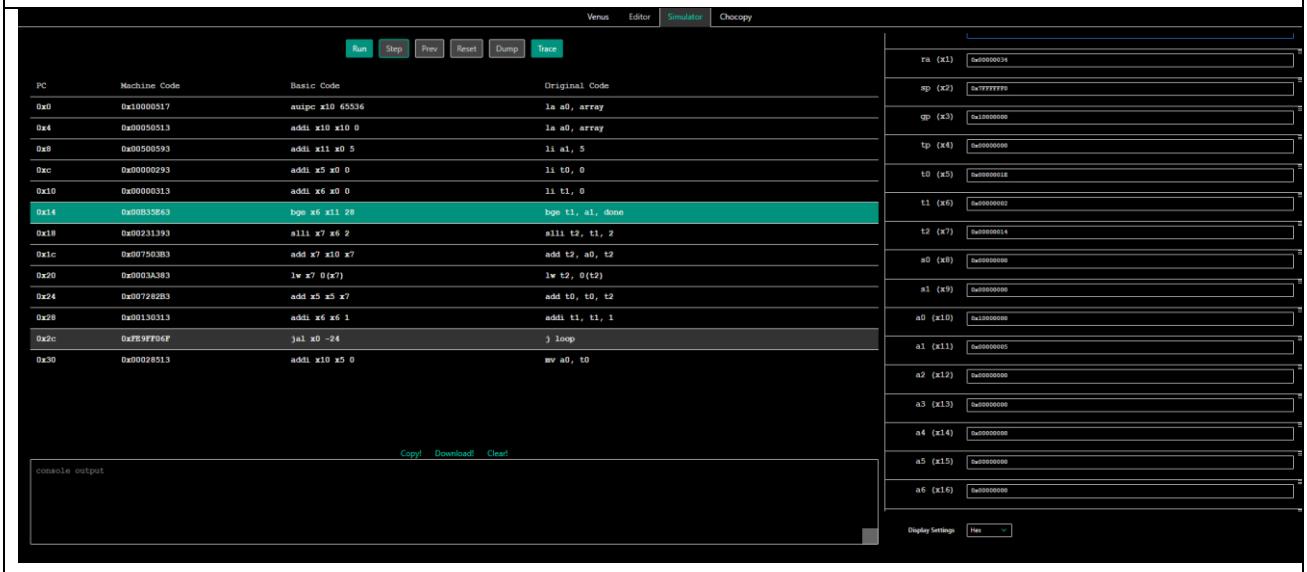
    slli t2, t1, 2
    add t2, a0, t2
    lw t2, 0(t2)

    add t0, t0, t2
    addi t1, t1, 1
```

```
    j loop


done:
    mv a0, t0
```

Output:



Task 5:

Description:

In this task, I initialized the **table of two** in the `.data` section.
I stored the table values from **2 to 20** in memory.
I then loaded each value using the lw instruction.
Each value was stored in a different RISC-V register.
Finally, I ran the program in **Venus Simulator** and checked the register values.

Code:

```
.data

table: .word 2, 4, 6, 8, 10, 12, 14, 16, 18, 20

.text

.globl main

main:

   la t0, table

   lw s0, 0(t0)

   lw s1, 4(t0)

   lw s2, 8(t0)

   lw s3, 12(t0)

   lw s4, 16(t0)
```

```
lw s5, 20(t0)

lw s6, 24(t0)

lw s7, 28(t0)

lw s8, 32(t0)

lw s9, 36(t0)
```

Output:

